



Universidade Federal de Santa Catarina
Centro Tecnológico - CTC

Robô Coletor de Bolas - Relatório Entrega 2

<https://github.com/ECA-UFSC-FLN/2026.1-G7-RoboColetorBolas>

André dos Santos Viegas
Ema da Silva Rodrigues
Pedro Augusto Costa
Vitor de Souza Barcala

Relatório do Trabalho Prático realizado no âmbito da Unidade Curricular Projeto Integrador, do
Departamento de Automação e Sistemas – DAS

Florianópolis, 21 de maio de 2026.

Declaramos que o presente trabalho/relatório é da nossa autoria e não foi utilizado previamente nou-
tro curso ou unidade curricular, desta ou de outra instituição. As referências a outros autores (afirmações,
ideias, pensamentos) respeitam escrupulosamente as regras da atribuição, e encontram-se devidamente
indicadas no texto e nas referências bibliográficas, de acordo com as normas de referência. Temos
consciência de que a prática de plágio e autoplágio constitui uma infração académica.

Resumo

O presente relatório descreve a segunda etapa do desenvolvimento de um sistema robótico destinado à coleta automática de bolas de tênis. Nesta fase, o projeto evoluiu da definição conceitual para a implementação dos principais módulos de software, incluindo captura de imagens, detecção de bolas, calibração geométrica, localização do robô por marcadores ArUco, planejamento de trajetória e controle.

O sistema utiliza visão computacional para identificar as bolas e estimar a posição e orientação do robô no plano da quadra. As coordenadas obtidas na imagem são convertidas para um referencial métrico por meio de homografia, permitindo que o planejador gere uma sequência de waypoints para a coleta. Também foi iniciada a montagem do protótipo físico, que será utilizado nas próximas etapas para validar a integração com o ESP32 e os motores.

Palavras-chave: robótica autônoma, visão computacional, coleta automática de bolas, detecção de objetos, ArUco, planejamento de trajetória.

Índice

1	Introdução	3
1.1	Contexto Tecnológico e Motivação	3
1.2	Empresa Parceira: Drillbot	3
1.3	Solução Proposta	4
2	Materiais e equipamentos	4
2.1	Módulo de Visão e Processamento	4
2.2	Robô Móvel e Atuadores	5
3	Funcionamento do Sistema	5
4	Requisitos do Sistema	6
4.1	Árvore de Funcionalidades	7
4.2	Requisitos Funcionais e Não Funcionais	7
5	Modelação do Sistema	10
6	Organização da equipe	12
7	Cronogramas da primeira entrega	12
8	Resultados Preliminares	13
8.1	Detecção de bolas	13
8.2	Correção de Distorção	14
9	Desenvolvimento realizado na segunda etapa	15
9.1	Resumo dos avanços obtidos	16
10	Arquitetura de software implementada	17
10.1	Visão geral da arquitetura	17
10.2	Módulos principais	18
10.3	Comunicação entre processos	19

11 Calibração geométrica e transformação de perspectiva	19
11.1 Calibração por pontos de referência	20
11.2 Conversão de pixels para coordenadas métricas	20
11.3 Armazenamento da calibração	20
11.4 Resultado da retificação	21
12 Detecção de bolas por visão computacional	21
12.1 Pipeline de detecção	22
12.2 Modelo de detecção	22
12.3 Resultados obtidos	23
12.4 Integração com o planejamento	24
13 Localização e orientação do robô com ArUco	24
13.1 Disposição dos marcadores no robô	24
13.2 Cálculo da posição e orientação	25
13.3 Integração com o sistema de controle	25
14 Planejamento de trajetória	25
14.1 Construção do estado do sistema	26
14.2 Modos de operação	26
14.3 Planejamento global da rota	27
15 Controle do robô	27
15.1 Entradas do controlador	28
15.2 Estratégia de controle	28
15.3 Controle angular	29
15.4 Envio de comandos e integração com o protótipo	29
16 Configuração e depuração	30
16.1 Parâmetros configuráveis	30
16.2 Ferramentas de suporte ao desenvolvimento	30
17 Execução integrada do sistema	31
18 Montagem do protótipo físico	31
18.1 Estado atual da montagem	31
18.2 Integração prevista com o software	33
19 Testes realizados e validação parcial	33
20 Cronogramas da segunda entrega	34
20.1 Atividades realizadas entre a primeira e a segunda entrega	35
20.2 Atividades previstas até a entrega final	35
21 Dificuldades encontradas	35
22 Próximas etapas	36
23 Conclusão da segunda entrega	37

1 Introdução

1.1 Contexto Tecnológico e Motivação

A evolução contínua da robótica e dos sistemas autônomos tem impulsionado a criação de soluções inovadoras para automatizar tarefas repetitivas em diversos setores. No domínio desportivo, especialmente no tênis, têm surgido oportunidades significativas para o desenvolvimento de tecnologias que otimizem o treino, reduzam o esforço manual e aumentem a eficiência operacional das quadras.

Entre essas tarefas encontra-se a coleta de bolas após sessões de treino. Embora simples, trata-se de uma atividade que consome tempo, interrompe o ritmo do praticante e exige deslocamentos constantes pela quadra. Em ambientes profissionais ou de elevada rotatividade, esse processo constitui um fator limitante, prejudicando a fluidez dos treinos e aumentando o *down time* entre sessões. Além disso, em treinos de alta intensidade, a acumulação de bolas no piso pode comprometer o desempenho e até gerar riscos de queda.

O presente projeto insere-se num cenário de inovação, buscando aplicar técnicas modernas de robótica e automação para resolver um problema real e relevante no cotidiano do tênis.

1.2 Empresa Parceira: Drillbot

A Drillbot é uma startup sediada em Florianópolis–SC dedicada ao desenvolvimento de tecnologias inovadoras para o ambiente desportivo, com foco em automação aplicada ao tênis. A empresa opera através da locação de máquinas de lançamento de bolas instaladas em clubes e condomínios, permitindo que os usuários agendem minutos de treino e configurem seus exercícios diretamente através de um aplicativo.

A tecnologia atual da Drillbot baseia-se em uma máquina capaz de armazenar até 130 bolas e lançá-las com intervalos ajustáveis entre 2 e 4 segundos, resultando em um ciclo de treino completo de aproximadamente 7 minutos. No entanto, após o lançamento de todas as bolas, o processo de recolha permanece manual: um cano coletor de 75 mm é utilizado para coletar individualmente cada bola, exigindo cerca de 5 minutos para completar a coleta das mesmas 130 bolas. Este tempo de coleta equivale a quase todo o ciclo de lançamento, criando um período significativo de inatividade da quadra e impactando negativamente a experiência do usuário.

Reconhecendo esta limitação, a Drillbot procurou a UFSC para o desenvolvimento de uma solução autônoma, capaz de reduzir drasticamente o tempo de coleta e eliminar a necessidade de intervenção manual. A empresa também disponibilizou acesso a uma quadra real no Centro de Inovação TXM / Fundação Fiesp para testes e validações práticas, além de referências comerciais já existentes no mercado internacional.



Figura 1: Interface da aplicação da Drillbot para controle da máquina lançadora de bolas. [1]

1.3 Solução Proposta

A solução estudada consiste na implementação de um sistema integrado composto por:

- um módulo de visão computacional capaz de identificar e localizar as bolas em quadra;
- um robô móvel autônomo responsável por percorrer a quadra e coletar as bolas de forma eficiente.

O sistema de visão baseia-se na captura de imagens da área de jogo através de uma câmera posicionada estrategicamente na quadra. A partir dessas imagens, algoritmos de visão computacional serão aplicados para identificar com precisão a localização das bolas espalhadas pela superfície. Após o mapeamento das coordenadas, o sistema gerará uma trajetória otimizada que será transmitida ao robô, orientando o seu deslocamento para a coleta eficiente das bolas.

Considerando a elevada complexidade técnica envolvida no desenvolvimento simultâneo de um robô e de um sistema de visão robusto, a equipe decidiu concentrar os esforços iniciais no processamento de imagem e na geração de trajetórias. O foco principal será garantir que a detecção das bolas, o mapeamento espacial e a lógica de navegação operem de forma confiável.

Para viabilizar testes práticos e validar o funcionamento sistêmico geral os materiais e componentes físicos do robô serão emprestados pelo professor e parceiros da UFSC, permitindo a integração e demonstração do fluxo completo da solução.

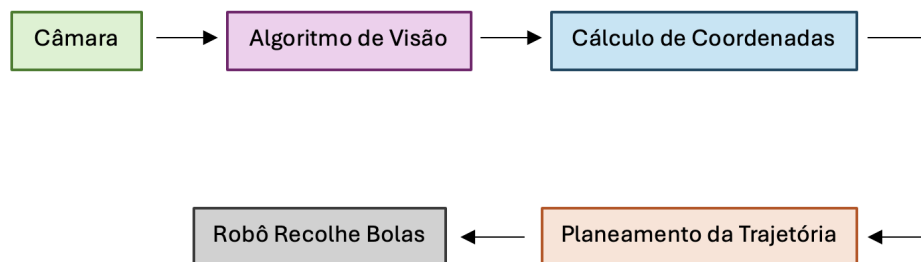


Figura 2: Fluxo geral de funcionamento do sistema proposto para coleta automática de bolas.

2 Materiais e equipamentos

O desenvolvimento do sistema requer a combinação de um módulo de visão computacional, um sistema de processamento responsável pelo cálculo das trajetórias e uma plataforma robótica capaz de executar os movimentos planejados. Nesta fase inicial, a equipe optou por utilizar componentes já disponíveis no laboratório e equipamentos cedidos pela UFSC, assegurando que o projeto avance de forma eficiente e compatível com os recursos e o cronograma estabelecidos.

Os materiais estão organizados em dois módulos principais: (i) aquisição e processamento de imagem; e (ii) robô móvel e atuadores.

2.1 Módulo de Visão e Processamento

Este módulo é responsável pela captura das imagens da quadra, detecção das bolas e cálculo das coordenadas necessárias para o planejamento da trajetória. Os componentes definidos para esta etapa incluem:

- **Câmera para captura de imagem:** Inicialmente será utilizada a câmera de um celular (iPhone 16), permitindo a criação de um *dataset* diversificado para treino e validação dos algoritmos de

visão computacional. Em etapas posteriores, está prevista a substituição por uma câmera dedicada para microcontroladores, com melhor estabilidade, ângulo de visão adequado e capacidade de operação contínua.

- **Placa de processamento (*Raspberry Pi*):** O processamento das imagens, incluindo detecção de bolas, filtragem, cálculo de coordenadas e geração de trajetórias otimizadas, será realizado numa *Raspberry Pi* disponibilizada pelo professor. A placa contará com um módulo acelerador de IA, permitindo a execução eficiente de modelos de visão computacional em tempo real. Numa fase inicial do projeto, será utilizado um computador para este fim.
- **Software e bibliotecas:** A implementação do algoritmo fará uso de bibliotecas como OpenCV, NumPy e frameworks de inferência de IA suportados pelo acelerador. Este conjunto permitirá analisar as imagens capturadas, identificar as bolas e convertê-las em representações métricas da quadra.

2.2 Robô Móvel e Atuadores

O segundo módulo envolve a plataforma robótica responsável pelo deslocamento físico na quadra e pela coleta das bolas. Os materiais utilizados incluem:

- **Protótipo de robô móvel:** Será utilizado um robô já disponível no laboratório da UFSC, cedido para o desenvolvimento do projeto. O protótipo será adaptado para integrar os sinais de navegação provenientes do módulo de visão e para permitir testes práticos do sistema completo.
- **Placa de controle de motores (*drivers*):** A locomoção do robô será controlada por um *ESP32* ou *Arduino*. Estes controladores converterão os comandos enviados pela *Raspberry Pi* ou pelo computador em sinais adequados aos motores, permitindo deslocamento preciso pelas coordenadas calculadas.
- **Sensores e componentes auxiliares:** Dependendo da necessidade durante o desenvolvimento, poderão ser integrados sensores adicionais, como encoders para odometria, IMU para estabilização do movimento e elementos mecânicos para auxiliar no mecanismo de recolha. Estes sensores, em conjunto com a visão computacional, irão contribuir para a estimação da posição do robô.

Em conjunto, estes módulos constituem a base do sistema, conectando a análise das imagens ao planejamento e à execução dos movimentos do robô. Essa abordagem modular permite validar o funcionamento do sistema passo a passo.

3 Funcionamento do Sistema

O funcionamento operacional do robô coletor é organizado em uma sequência estruturada de etapas que integram a aquisição de dados, o processamento inteligente das informações e a atuação física em campo. O fluxo completo é descrito a seguir:

1. **Captura da Imagem:** O ciclo inicia-se com uma câmera externa posicionada em um ponto estratégico da quadra, permitindo a obtenção de uma visão ampla e atualizada da área de detecção.
2. **Transmissão dos Dados:** O *frame* capturado é enviado para a unidade de processamento, que pode ser um computador ou uma *Raspberry Pi*, responsável por executar os algoritmos principais do sistema.

3. **Processamento de Visão Computacional:** A imagem recebida é processada por um algoritmo de detecção treinado para identificar as bolas de tênis no cenário, segmentando visualmente os objetos de interesse.
4. **Correção e Remoção de Distorção:** Para garantir alta precisão espacial, o sistema aplica técnicas de calibração de câmera e correção geométrica, eliminando distorções da lente e ajustando a perspectiva da imagem.
5. **Cálculo das Coordenadas:** Com a imagem projetada, o sistema converte as detecções em coordenadas 2D (Referencial (x, y)) correspondentes ao plano real da quadra, permitindo a localização exata de cada bola.
6. **Planejamento do Percurso:** Com base nas coordenadas obtidas, um algoritmo de planejamento determina a sequência de recolha mais eficiente, otimizando o percurso total a ser percorrido pelo robô.
7. **Transmissão da Trajetória:** O plano final de deslocamento é enviado sem fios ao controlador do robô, que interpreta cada comando para execução da rota calculada.
8. **Deslocamento e Execução:** O robô move-se até as posições indicadas, recolhe as bolas e, ao concluir a tarefa, retorna para um estado de espera até o início de um novo ciclo operacional.

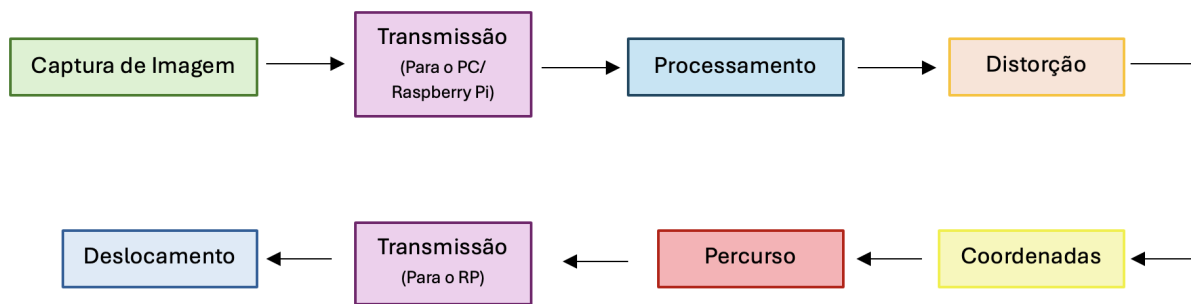


Figura 3: Diagrama das etapas de processamento de imagem e navegação.

4 Requisitos do Sistema

A definição clara dos requisitos é fundamental para orientar o desenvolvimento do sistema e garantir que todas as funcionalidades previstas sejam implementadas de forma coerente com os objetivos do projeto.

Deste modo, a seguinte seção apresenta os requisitos identificados até o momento, organizados de forma a refletir a estrutura funcional do robô coletor.

Os requisitos funcionais descrevem o que o sistema deve realizar, enquanto os requisitos não funcionais detalham critérios de desempenho, precisão, segurança e operação que cada funcionalidade deve atender. Para contextualizar estes elementos e mostrar a relação entre as diferentes partes do sistema, apresenta-se também a árvore de funcionalidades desenvolvida pela equipe.

4.1 Árvore de Funcionalidades

A Figura 4 apresenta a árvore de funcionalidades desenvolvida pela equipe, organizada em três grandes módulos: *Percepção e Mapeamento*, *planejamento e Navegação* e *Atuação e Monitorização*. Esta representação hierárquica descreve como cada funcionalidade de alto nível se desdobra em subfunções específicas que compõem o comportamento completo do sistema.

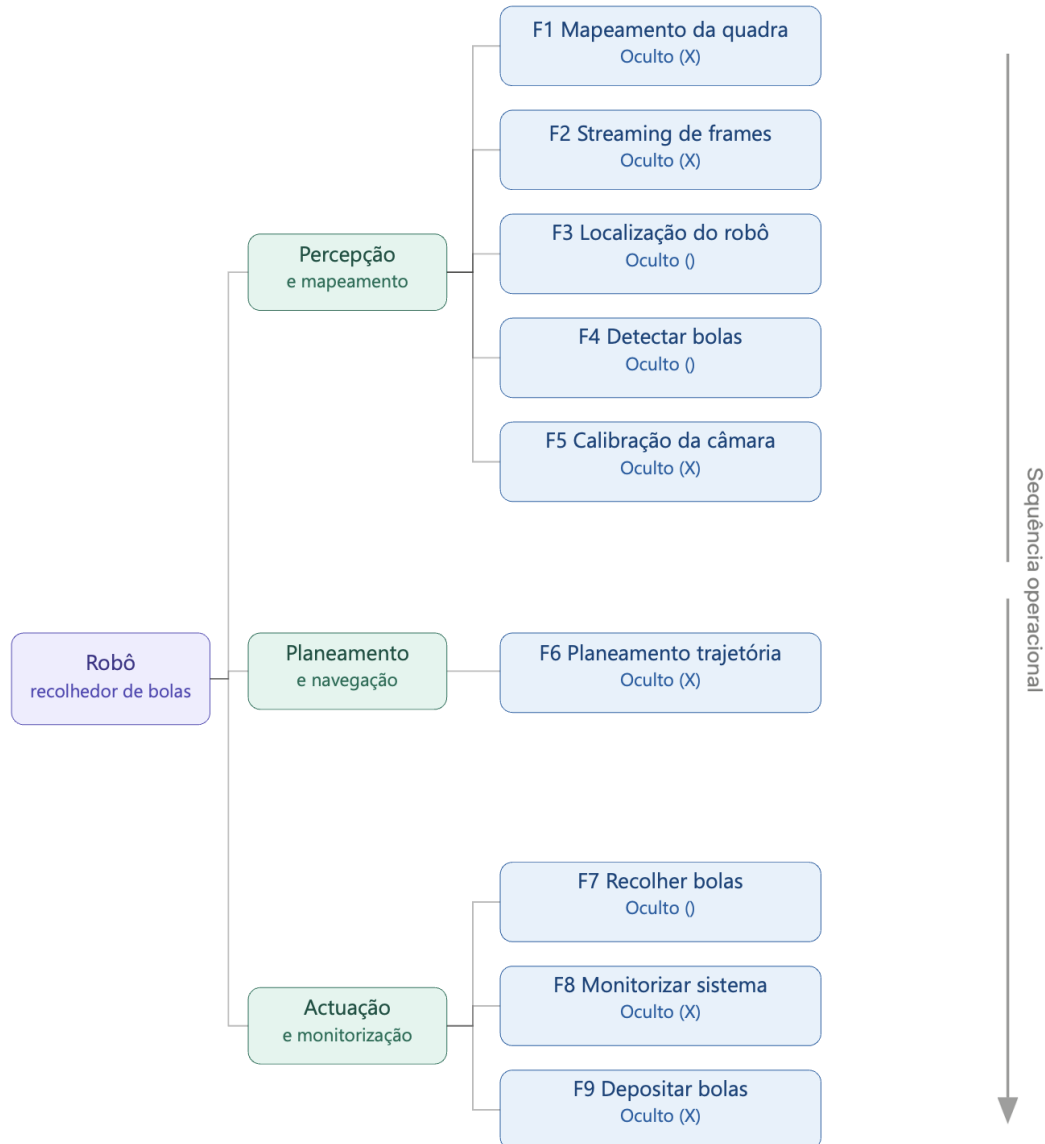


Figura 4: Árvore de funcionalidades do sistema de coleta automática de bolas.

4.2 Requisitos Funcionais e Não Funcionais

A seguir apresentam-se as tabelas de requisitos funcionais (RF) e seus correspondentes requisitos não funcionais (RNF). Cada funcionalidade é descrita com o seu propósito, seguida das restrições e características que devem ser atendidas para garantir o correto desempenho do sistema.

Nota-se que alguns requisitos incluem variáveis indicadas pelo símbolo "X". Estes valores serão definidos posteriormente, durante a fase de testes e calibração do protótipo, quando for possível medir com precisão parâmetros como resolução mínima, erros admissíveis, tempos máximos de processamento

e limites operacionais. Como este é o primeiro relatório do semestre, tais valores permanecem em aberto até que os dados experimentais estejam disponíveis.

RF 1.1: Mapeamento da Quadra (Configuração Inicial)					Oculto (X)
Descrição: Representar a quadra como um mapa 2D, registrando dimensões e zonas proibidas.					
Requisitos Não Funcionais					
Código	Nome	Restrição	Categoria	Desejável	Permanente
NF1.1.1	Representação do mapa	O mapa deve ser representado como uma grelha de ocupação com resolução mínima de Xcm.	Especificação	(X)	(X)
NF1.1.2	Configurabilidade	As dimensões e zonas da quadra devem ser configuráveis para diferentes tipos.	Interface	(X)	()
NF1.1.3	Atualização dinâmica	O mapa deve ser atualizado em tempo real quando novos obstáculos temporários forem detectados.	Performance	()	(X)

RF 1.2: Streaming e Seleção de Frames					Oculto (X)
Descrição: O sistema deve estabelecer uma conexão de streaming em tempo real entre a câmera (celular ou câmera fixa) e o computador de processamento, permitindo selecionar a fonte de vídeo desejada. Deve ainda capturar e extrair frames individuais do stream para processamento de visão computacional (detecção de bolas).					
Requisitos Não Funcionais					
Código	Nome	Restrição	Categoria	Desejável	Permanente
NF1.2.1	Latência do Stream	A latência entre captura e recepção no computador deve ser inferior a Xms.	Performance	(X)	(X)
NF1.2.2	Resolução mínima	O stream deve suportar resolução mínima de X pixels para garantir a detecção correta das bolas.	Especificação	()	(X)
NF1.2.3	Protocolo de comunicação	O sistema deve usar um protocolo padrão, como o X.	Especificação	()	(X)
NF1.2.4	Estabilidade da ligação	O sistema deve reconectar automaticamente em caso de perda de sinal.	Segurança	(X)	(X)

RF 1.3: Localização do Robô na Quadra (Odometria)					Oculto ()
Descrição: O sistema deve estimar a posição e orientação do robô na quadra em tempo real, combinando dados de encoders das rodas, IMU e/ou referências visuais.					
Requisitos Não Funcionais					
Código	Nome	Restrição	Categoria	Desejável	Permanente
NF1.3.1	Precisão de localização	O erro de posição acumulado não deve ultrapassar os Xcm por metro percorrido.	Performance	(X)	(X)
NF1.3.2	Frequência de atualização	A posição do robô deve ser atualizada a cada X segundos.	Performance	(X)	(X)
NF1.3.3	Recuperação de deriva	O sistema deve usar referências fixas da quadra para corrigir deriva de odometria.	Especificação	()	(X)

RF 1.4: Detectar e Localizar Bolas de Tênis					Oculto ()
Descrição: O sistema deve detectar bolas de tênis no campo visual da câmera e calcular a posição 2D (coordenadas na quadra) de cada bola, usando técnicas de visão computacional (ex: detecção por cor, forma ou modelo treinado).					
Requisitos Não Funcionais					
Código	Nome	Restrição	Categoria	Desejável	Permanente
NF1.4.1	Taxa de detecção	O sistema deve detectar bolas com taxa de acerto em condições normais de iluminação.	Performance	(X)	()
NF1.4.2	Diferenciação de objetos	O sistema deve distinguir bolas de tênis de outros objetos presentes na quadra (ex: sapatilhas, raquetes).	Interface	()	(X)
NF1.4.3	Latência de detecção	O tempo entre captura da imagem e obtenção da posição da bola deve ser inferior a X segundos.	Performance	(X)	(X)

RF 1.5: Calibração e Correção de Distorção da Câmera					Oculto (X)
Descrição: O sistema deve calibrar a câmera e corrigir distorções ópticas da lente antes da operação, garantindo que as posições detectadas na imagem correspondam às posições reais na quadra.					
Requisitos Não Funcionais					
Código	Nome	Restrição	Categoria	Desejável	Permanente
NF1.5.1	Qualidade da calibração	O erro residual de calibração deve ser inferior a X pixels após a correção de distorção.	Especificação	(X)	(X)
NF1.5.2	Repetibilidade	O processo de calibração deve produzir resultados consistentes em diferentes sessões.	Performance	()	(X)

RF 1.6: Planejamento de Trajetória					Oculto (X)
Descrição: O sistema deve calcular a sequência ótima de bolas a coletar e o percurso mais eficiente (menor distância/tempo), evitando obstáculos fixos e dinâmicos presentes na quadra.					
Requisitos Não Funcionais					
Código	Nome	Restrição	Categoria	Desejável	Permanente
NF1.6.1	Algoritmo de otimização	O percurso deve ser planejado usando um algoritmo que minimize a distância percorrida.	Especificação	(X)	(X)
NF1.6.2	Tempo de cálculo	O tempo de geração da trajetória completa deve ser inferior a X segundos.	Performance	(X)	()
NF1.6.3	Replanificação	O sistema deve ser capaz de recalcular o percurso em tempo real se um novo obstáculo for detectado.	Performance	(X)	(X)

RF 1.7: Coletar Bolas do Chão					Oculto ()
Descrição: O robô deve apanhar as bolas de tênis do chão utilizando o mecanismo de recolha (ex: rolos, aspiração ou garra) e armazená-las no reservatório interno ao aproximar-se da posição da bola.					
Requisitos Não Funcionais					
Código	Nome	Restrição	Categoria	Desejável	Permanente
NF1.7.1	Taxa de recolha	O robô deve coletar com sucesso pelo menos X% das bolas que abordar.	Performance	(X)	(X)
NF1.7.2	Capacidade de armazenamento	O reservatório deve ter capacidade para armazenar pelo menos X bolas de tênis.	Especificação	(X)	(X)
NF1.7.3	Tempo de recolha	O tempo total para coletar todas as bolas da quadra não deve exceder X minutos.	Segurança	()	(X)

RF 1.8: Monitorar e Reportar o Estado do Sistema					Oculto (X)
Descrição: O sistema deve monitorar continuamente o estado operacional do robô (bateria, reservatório, sensores, erros) e disponibilizar essa informação ao operador em tempo real via interface.					
Requisitos Não Funcionais					
Código	Nome	Restrição	Categoria	Desejável	Permanente
NF1.8.1	Alerta de bateria baixa	O sistema deve emitir alerta quando a carga da bateria for inferior a X%.	Segurança	(X)	(X)
NF1.8.2	Autonomia da bateria	O robô deve operar continuamente sempre que estiver com carga completa por pelo menos X horas.	Performance	(X)	(X)
NF1.8.3	Interface de monitoramento	O operador deve poder consultar o estado do robô em tempo real via app ou display.	Interface	(X)	()
NF1.8.4	Registro de operação	O sistema deve salvar logs de cada sessão (bolas recolhidas, distância percorrida, erros).	Especificação	()	(X)

RF 1.9: Depositar Bolas no Ponto de Entrega					Oculto (X)
Descrição: O robô deve navegar até o ponto de entrega configurado e depositar as bolas recolhidas quando o reservatório estiver cheio ou por comando do operador.					
Requisitos Não Funcionais					
Código	Nome	Restrição	Categoria	Desejável	Permanente
NF1.9.1	Configuração do ponto	O ponto de depósito deve ser configurável antes do início da operação.	Interface	(X)	(X)
NF1.9.2	Precisão de descarga	O robô deve alinhar-se ao ponto de entrega com erro inferior a X cm.	Performance	(X)	(X)

De forma geral, os requisitos apresentados nesta seção estabelecem as bases técnicas e operacionais para o desenvolvimento do sistema de coleta automática de bolas.

Embora alguns parâmetros permaneçam em aberto até à realização de testes práticos, o conjunto atual fornece uma visão clara das funcionalidades pretendidas, das limitações a considerar e dos critérios mínimos de desempenho.

Estes requisitos servirão como referência central para as próximas etapas do projeto, orientando tanto a implementação incremental como a futura validação do protótipo em ambiente real.

5 Modelação do Sistema

Nesta seção são apresentados o diagrama de atividade e o diagrama de sequência do sistema proposto, com o objetivo de representar o seu funcionamento global e a interação entre os diferentes componentes. O diagrama de atividade descreve o fluxo lógico das operações envolvidas na recolha das bolas, enquanto o diagrama de sequência evidencia a troca de mensagens entre os módulos do sistema ao longo do tempo.

A Figura 5 apresenta o diagrama de atividade do sistema, ilustrando as principais etapas desde a captura da imagem até à execução da recolha pelo robô.

Neste contexto, destacam-se processos como o processamento de imagem, o cálculo das coordenadas e o planeamento da trajetória, que conduzem à atuação do robô em campo.

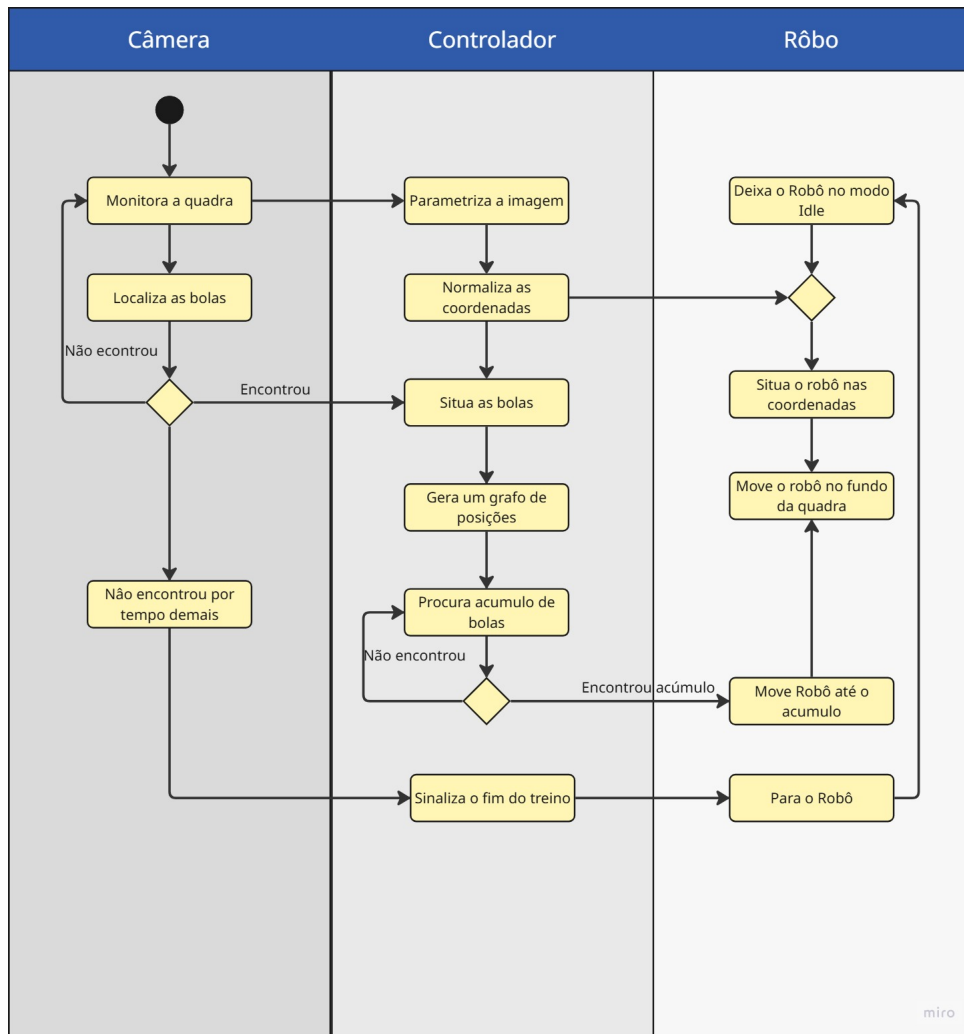


Figura 5: Diagrama de atividade do sistema de coleta automática de bolas.

A Figura 6 apresenta o diagrama de sequência do sistema, evidenciando a interação temporal entre os diferentes módulos.

É possível observar a comunicação entre a câmera, a unidade de processamento e o robô, desde a aquisição da imagem até à execução do percurso de recolha.

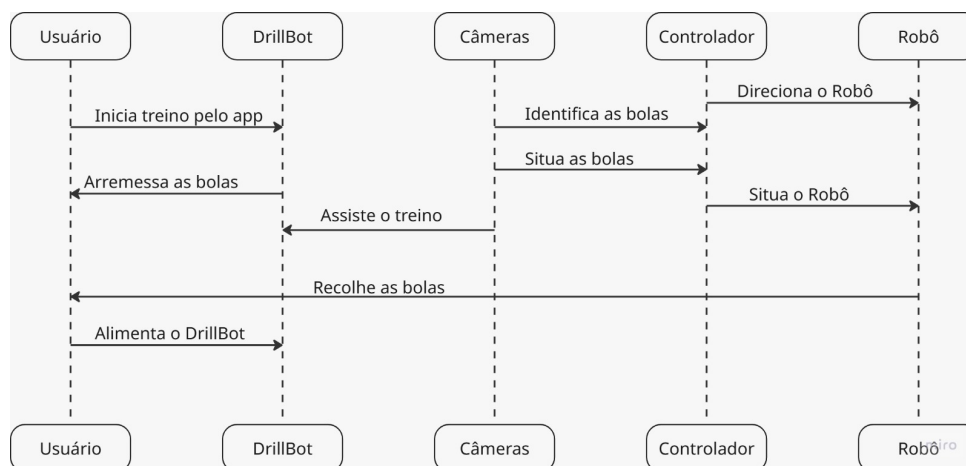


Figura 6: Diagrama de sequência do sistema.

6 Organização da equipe

Nesta seção apresenta-se a divisão de responsabilidades entre os elementos da equipe, tendo em conta as diferentes áreas do projeto.

A distribuição das tarefas foi definida de forma a equilibrar a carga de trabalho e aproveitar as competências individuais de cada elemento.

- André: Desenvolvimento do módulo de visão computacional para detecção das bolas de tênis e do robô.
- Ema: Algoritmo de calibração da câmera e transformação de perspectiva para mapeamento das coordenadas da quadra.
- Pedro: Planejamento de trajetória e algoritmos de navegação.
- Vítor: Integração do sistema e controle do robô.

A divisão de tarefas poderá ser ajustada ao longo do projeto, conforme a evolução do sistema e as necessidades identificadas durante o desenvolvimento.

7 Cronogramas da primeira entrega

Nesta seção são apresentados os cronogramas definidos na primeira entrega do projeto, incluindo as atividades realizadas na fase inicial e o planejamento preliminar das etapas futuras.

O desenvolvimento do projeto está organizado em 12 semanas, abrangendo desde a concepção e levantamento de requisitos até à integração final e testes em campo, com o objetivo de garantir uma evolução estruturada e o cumprimento dos prazos estabelecidos.

Os cronogramas detalhados nas Figuras 7 e 8 representam, respectivamente, as atividades realizadas na fase inicial e o planejamento para a conclusão do sistema.

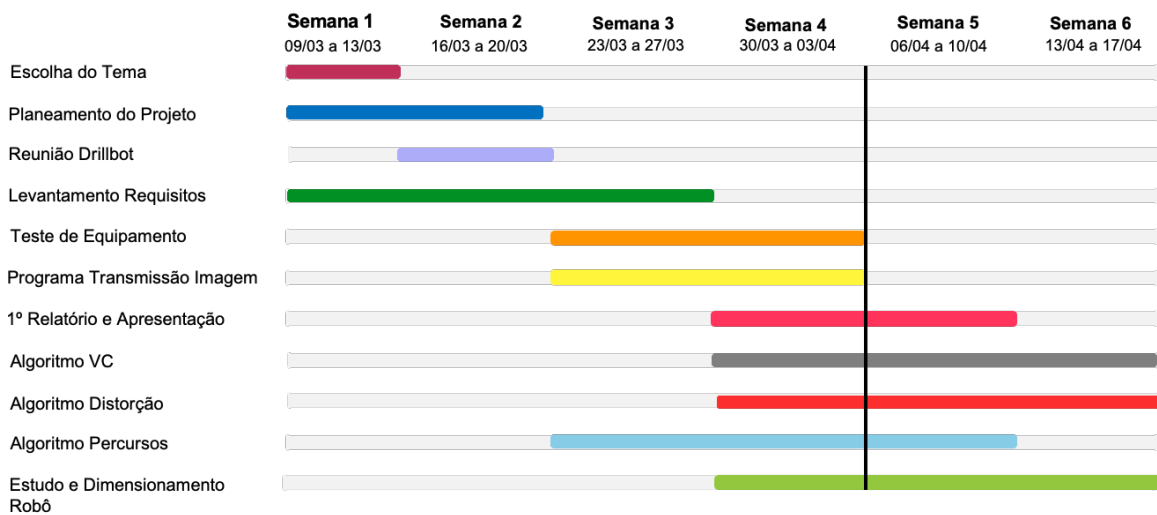


Figura 7: Cronograma das atividades atuais.

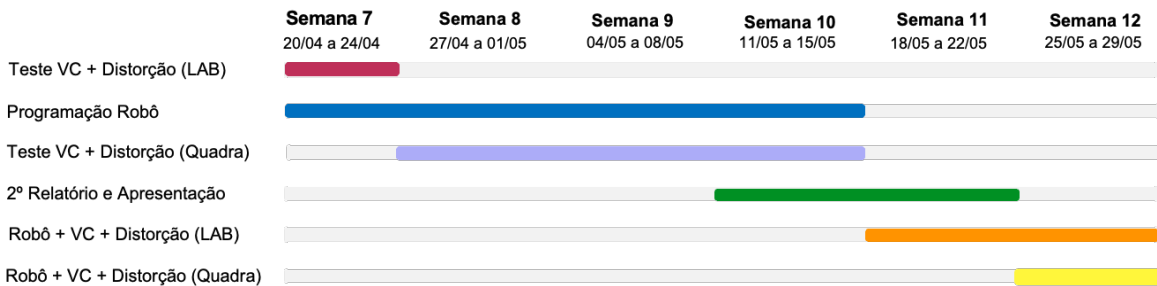


Figura 8: Cronograma das atividades previstas.

De forma geral, o cronograma foi definido de modo a permitir uma evolução progressiva do sistema, desde a fase de concepção até à validação final em ambiente real.

Este planeamento poderá ser ajustado ao longo do projeto, conforme os resultados obtidos e eventuais desafios técnicos identificados.

8 Resultados Preliminares

Nesta seção são apresentados os primeiros resultados obtidos durante a fase inicial do projeto, antes da integração completa dos módulos de software.

Estes resultados serviram como validação preliminar das abordagens de detecção de bolas e correção geométrica, sendo posteriormente aprofundados e integrados ao sistema nas etapas seguintes.

8.1 Detecção de bolas

Devido à reduzida dimensão do dataset inicial, composto por aproximadamente 100 imagens, o desempenho do modelo ainda se encontra aquém do desejado, apresentando limitações na identificação

consistente das bolas.

Com a expansão do dataset para cerca de 20 000 imagens, espera-se uma melhoria significativa na robustez e precisão do algoritmo.

A Figura 9 apresenta um resultado preliminar do algoritmo de detecção de bolas de tênis, desenvolvido com base em técnicas de visão computacional. É possível observar que o sistema identifica parcialmente as bolas presentes, embora ainda apresente falhas em cenários com maior complexidade visual.



Figura 9: Exemplo de detecção de bolas de tênis utilizando o modelo desenvolvido.

8.2 Correção de Distorção

A calibração da câmera é uma etapa essencial para garantir que as posições detectadas na imagem correspondam, com precisão, às posições reais na quadra. Câmeras de celulares, como a utilizada na fase inicial dos testes (iPhone 16), apresentam distorções radiais e tangenciais devido ao formato das lentes, o que provoca curvaturas nas extremidades e deformações na geometria da imagem.

As Figuras 10 e 11 ilustram o processo de correção de distorção aplicado à imagem capturada em laboratório. Na primeira, observa-se a imagem original, sem qualquer correção; na segunda, é possível ver a projeção corrigida, que aproxima a captura a uma vista superior projetada da quadra.

Esse processo é fundamental para garantir que a correspondência entre pixels da imagem e coordenadas reais seja consistente, reduzindo erros sistemáticos no cálculo das posições das bolas.



Figura 10: Imagem original sem correção de distorção.

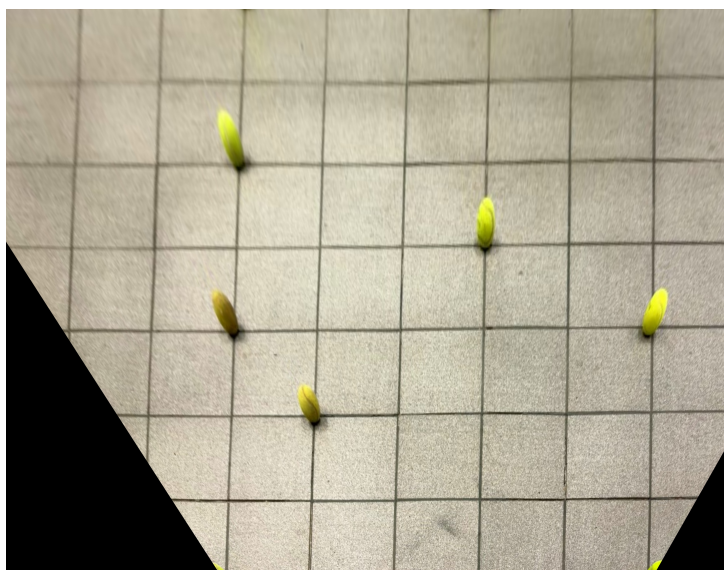


Figura 11: Imagem após processo de calibração e correção de distorção.

A partir da imagem corrigida, torna-se possível aplicar algoritmos de detecção e conversão de coordenadas com maior precisão, reduzindo erros acumulados na estimação da posição das bolas.

À medida que o projeto avançar para testes em quadra real e com câmeras dedicadas, este processo será refinado para garantir maior estabilidade geométrica e consistência métrica.

9 Desenvolvimento realizado na segunda etapa

Na primeira entrega, o projeto encontrava-se numa fase de definição da arquitetura, levantamento de requisitos e validação preliminar dos módulos de visão computacional e correção geométrica. Nesta segunda etapa, o foco passou a ser a implementação dos principais módulos do sistema e a sua integração progressiva em um fluxo funcional.

O desenvolvimento realizado concentrou-se em quatro frentes principais: percepção visual, calibração geométrica, localização do robô e preparação da integração com a plataforma física. O módulo de visão computacional foi consolidado, permitindo a detecção das bolas de tênis e a identificação do

robô na imagem. Paralelamente, o processo de correção de perspectiva foi aprimorado, possibilitando a conversão das posições detectadas em pixels para coordenadas métricas no plano da quadra.

Outro avanço relevante foi a implementação da localização do robô por meio de dois marcadores ArUco fixados na parte superior da plataforma, um alinhado à região frontal e outro à região traseira do robô. Esta disposição foi escolhida porque a câmera responsável pela captura das imagens será posicionada na parede lateral da quadra, com uma inclinação em relação ao plano do chão, permitindo visualizar a parte superior do robô durante a operação. Para garantir uma detecção estável dos marcadores, está prevista a utilização de uma superfície rígida e plana sobre o robô, possivelmente em cartão pluma, onde serão fixadas as imagens dos ArUco.

Com essa configuração, tornou-se possível estimar não apenas a posição do robô no plano da quadra, mas também a sua orientação, calculada a partir do alinhamento entre os dois marcadores, informação essencial para o controle de movimento.

Além disso, foram desenvolvidos os módulos de planejamento de trajetória e controle, preparando o sistema para enviar comandos à plataforma robótica através de comunicação UDP com o ESP32.

A montagem física do protótipo também foi iniciada nesta etapa. Embora a integração completa com o robô ainda esteja em andamento, o software já se encontra estruturado para receber dados da câmera, processar as posições das bolas e do robô, calcular uma rota de coleta e gerar comandos de movimento.

9.1 Resumo dos avanços obtidos

A Tabela 1 apresenta uma comparação entre o estado dos principais módulos na primeira entrega e o estado atual do desenvolvimento.

Módulo	Estado na 1ª entrega	Estado na 2ª entrega
Detecção de bolas	Testes preliminares com desempenho limitado pelo tamanho reduzido do conjunto de dados.	Pipeline de visão computacional finalizado em sua versão atual, com detecção de bolas integrada ao sistema.
Correção de distorção e perspectiva	Correção inicial de distorção e estudo da transformação da imagem para uma vista superior.	Algoritmo de homografia aprimorado, permitindo converter coordenadas em pixels para coordenadas métricas.
Localização do robô	Funcionalidade prevista nos requisitos e na arquitetura do sistema.	Localização implementada com dois marcadores ArUco, permitindo estimar posição e orientação do robô.
Planejamento de trajetória	Planejamento descrito conceitualmente como parte da solução proposta.	Planejamento global implementado com algoritmo de vizinho mais próximo e otimização <i>2-opt</i> .
Controle do robô	Controle previsto como etapa futura de integração com a plataforma móvel.	Controlador implementado com cálculo de velocidade linear e angular, PID angular e envio por UDP ao ESP32.
Integração física	Utilização futura de um robô disponível no laboratório ou adaptado para o projeto.	Montagem do protótipo físico em andamento, com preparação para integração com o software desenvolvido.

Tabela 1: Evolução dos principais módulos do sistema entre a primeira e a segunda entrega.

Os avanços apresentados mostram que o projeto passou de uma etapa de especificação e validação inicial para uma fase de implementação integrada. Nas próximas seções, são descritos com maior detalhe a arquitetura de software implementada, os métodos de calibração, a detecção das bolas, a localização do robô, o planejamento de trajetória e a estratégia de controle.

10 Arquitetura de software implementada

Nesta etapa, o sistema passou a ser organizado como um conjunto de módulos independentes, executados de forma coordenada. Cada módulo é responsável por uma etapa específica do funcionamento do robô, desde a captura das imagens até a geração dos comandos enviados à plataforma física.

Esta organização modular permite testar cada componente de forma isolada, facilita a depuração e torna a integração mais controlada. Além disso, permite que o sistema continue a ser desenvolvido mesmo enquanto a montagem física do robô ainda está em andamento.

10.1 Visão geral da arquitetura

A arquitetura implementada foi dividida em quatro camadas principais: captura, processamento de visão, planejamento e controle. A Figura 12 apresenta o fluxo geral de informação entre estas camadas.

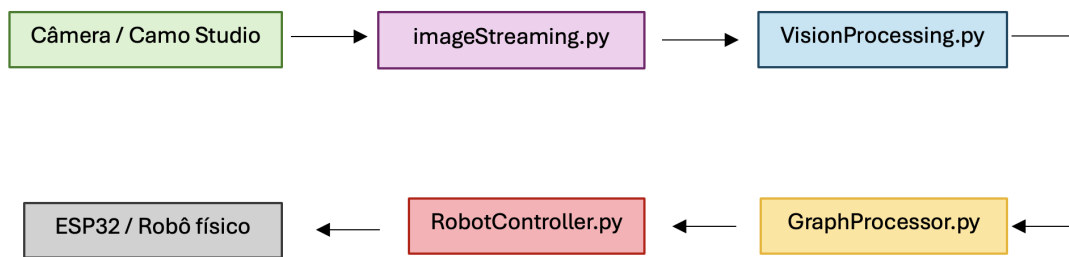


Figura 12: Fluxo geral da arquitetura de software implementada.

O fluxo inicia-se com a captura dos frames da câmera. Em seguida, as imagens são processadas para detectar as bolas de tênis e os marcadores ArUco do robô. As posições obtidas são convertidas para coordenadas métricas no plano da quadra e enviadas ao módulo responsável pelo estado do sistema. A partir destas informações, é definida a rota de coleta e são gerados os comandos de movimento para o robô.

10.2 Módulos principais

A Tabela 2 resume os principais módulos implementados e a função de cada um no sistema.

Módulo	Função principal
<code>imageStreaming.py</code>	Captura frames da câmera através do OpenCV. Possui modo de calibração, para envio de imagens ao retificador, e modo de produção, para envio contínuo ao módulo de visão.
<code>VisionProcessing.py</code>	Processa os frames recebidos, executando a detecção das bolas de tênis e a identificação dos marcadores ArUco do robô.
<code>retificador.py</code>	Realiza a calibração por homografia e converte coordenadas em pixels para coordenadas métricas no plano da quadra.
<code>GraphProcessor.py</code>	Acumula detecções de múltiplos frames, organiza o estado atual do sistema e define o modo operacional.
<code>BallCollectionPlanner.py</code>	Calcula a sequência de bolas a coletar no modo global, utilizando vizinho mais próximo e otimização <i>2-opt</i> .
<code>RobotController.py</code>	Recebe o estado do sistema, calcula comandos de velocidade linear e angular e prepara o envio ao ESP32 por UDP.
<code>MasterControl.py</code>	Inicializa e gerencia os processos principais do sistema.
<code>bolas_log.py</code>	Centraliza o registro de eventos, avisos, erros e mensagens de debug.
<code>debug_console.py</code>	Apresenta as mensagens de debug em um console separado.

Tabela 2: Principais módulos implementados no sistema.

10.3 Comunicação entre processos

A comunicação entre os módulos foi organizada através de portas locais e envio de mensagens. Esta separação permite que os processos sejam executados em paralelo, mantendo o fluxo de dados entre captura, visão, planejamento e controle.

As principais ligações de comunicação implementadas são:

- `imageStreaming.py` → `VisionProcessing.py` – **porta 6000**: envio dos frames capturados pela câmera para o módulo de visão computacional.
- `VisionProcessing.py` → `GraphProcessor.py` – **porta 6020**: envio das detecções das bolas e das informações de localização do robô.
- `GraphProcessor.py` → `RobotController.py` – **porta 6021**: envio do estado atual do sistema, incluindo alvo, waypoint, fase de operação e modo operacional.
- `RobotController.py` → **ESP32 – UDP**: envio dos comandos de movimento, incluindo velocidade linear e velocidade angular.
- **Módulos do sistema** → `debug_console.py` – **porta 6030**: envio de mensagens de debug para um console separado.
- `RobotController.py` – **porta 6014**: disponibilização de um mecanismo de *health-check*, utilizado para verificar se o controlador está ativo.

Um aspecto importante desta comunicação é o mecanismo de *handshake* entre o módulo de captura e o módulo de visão. O `imageStreaming.py` só envia um novo frame após receber a confirmação de que o processamento anterior foi liberado. Com isso, evita-se o acúmulo excessivo de imagens e melhora-se a estabilidade do pipeline.

11 Calibração geométrica e transformação de perspectiva

Na primeira entrega, a correção de distorção e perspectiva foi apresentada como um resultado preliminar do módulo de visão computacional, conforme mostrado na seção 8. Nesta segunda etapa, esse processo foi incorporado ao pipeline do sistema através do módulo `retificador.py`, passando a ser utilizado diretamente na conversão das detecções para coordenadas métricas.

O objetivo deste módulo é garantir que as posições detectadas na imagem, originalmente expressas em pixels, possam ser representadas em um referencial comum no plano da quadra. Essa conversão é necessária para que as bolas, os marcadores ArUco do robô e os waypoints de navegação sejam tratados no mesmo sistema de coordenadas.

Como a câmera será instalada na parede da quadra, com uma inclinação em relação ao plano do chão, a imagem capturada pode apresentar deformações de perspectiva. Para corrigir este efeito, o sistema utiliza uma transformação baseada em homografia, calculada a partir de quatro ou mais pontos de referência selecionados na imagem.

11.1 Calibração por pontos de referência

O processo de calibração é realizado a partir da seleção manual de quatro ou mais pontos de referência na imagem. Estes pontos correspondem a posições conhecidas da área de testes e definem o plano utilizado para a transformação de perspectiva.

A partir da correspondência entre os pontos seleccionados na imagem e as suas coordenadas reais, o sistema calcula a matriz de homografia. Essa matriz permite transformar pontos do referencial da imagem, expresso em pixels, para o referencial métrico da quadra, expresso em metros.

A Figura 13 ilustra o processo de seleção dos oito pontos de referência na imagem original.



Figura 13: Seleção dos oito pontos de referência utilizados para o cálculo da homografia.

11.2 Conversão de pixels para coordenadas métricas

Após o cálculo da matriz de homografia, cada ponto detectado na imagem pode ser projetado para o plano da quadra. Assim, uma bola ou marcador detectado em coordenadas de imagem (u, v) passa a ser representado por uma posição (x, y) no referencial métrico.

Esta conversão é fundamental para a integração entre visão computacional, planejamento e controle. Sem esta etapa, o sistema apenas saberia onde os objetos aparecem na imagem, mas não conseguiria calcular distâncias reais, definir trajetórias ou gerar comandos coerentes para o robô.

Além da matriz de homografia, o sistema calcula uma relação de escala em pixels por metro, denominada PPM. Este valor auxilia na interpretação métrica da imagem corrigida e permite validar se a transformação obtida é compatível com as dimensões reais da área observada.

11.3 Armazenamento da calibração

Para permitir que a calibração seja reutilizada durante a operação do sistema, os parâmetros obtidos são guardados em um arquivo JSON. Este arquivo contém, entre outras informações, a matriz de homografia e os parâmetros necessários para converter as coordenadas da imagem para o plano da quadra.

No sistema atual, o resultado da calibração é armazenado no seguinte caminho:

```
resultados/calibracao/homografia_calibracao.json
```

Esta abordagem evita que o processo de calibração precise ser repetido a cada execução, desde que a posição da câmera e a área de operação permaneçam inalteradas. Caso a câmera seja deslocada ou a área de testes seja modificada, uma nova calibração deve ser realizada.

11.4 Resultado da retificação

Após o cálculo da homografia, o sistema passa a gerar uma representação corrigida da área de operação. Esta imagem retificada aproxima a visualização de uma vista superior, facilitando a verificação visual das posições detectadas e da coerência da transformação geométrica.

A Figura 14 apresenta a mesma etapa de retificação com a sobreposição das posições detectadas e das coordenadas métricas estimadas. Esta visualização permite verificar se a transformação obtida é coerente com o referencial real da área de testes.

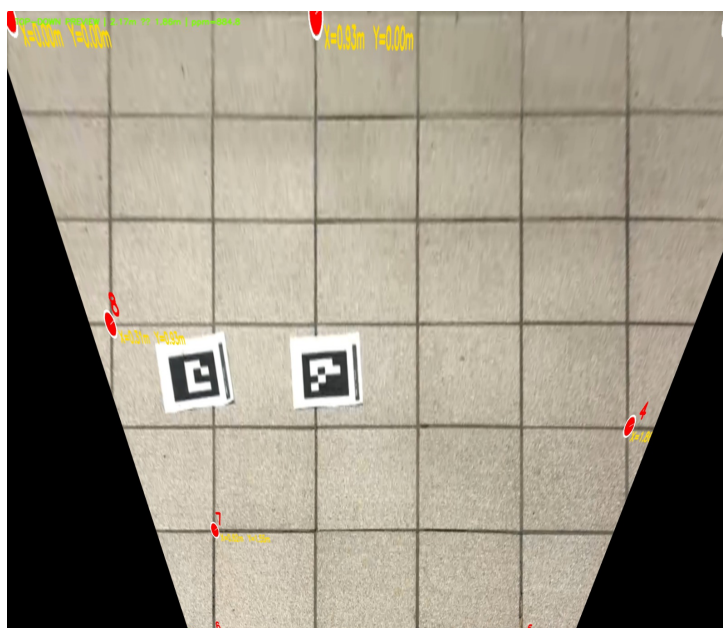


Figura 14: Resultado da homografia com indicação dos pontos detectados e das coordenadas métricas estimadas.

Com esta melhoria, a calibração deixa de ser apenas uma etapa visual de correção da imagem e passa a integrar diretamente o funcionamento do sistema. As coordenadas corrigidas são utilizadas pelos módulos de visão, planejamento e controle, permitindo que a posição das bolas e do robô seja tratada de forma consistente no mesmo referencial métrico.

12 Detecção de bolas por visão computacional

Nesta etapa, o módulo de detecção de bolas foi consolidado e integrado ao fluxo principal do sistema. Enquanto na seção 8 foram apresentados testes ainda preliminares, com um conjunto reduzido de imagens, nesta fase o objetivo passou a ser transformar a detecção em uma etapa funcional do pipeline de operação.

O módulo de visão computacional recebe os frames capturados pela câmera, identifica as bolas de tênis presentes na imagem e disponibiliza as suas posições para os módulos seguintes. Após a detecção

ção, as coordenadas obtidas são convertidas para o referencial métrico da quadra através do módulo de calibração geométrica, permitindo que sejam utilizadas no planejamento da trajetória.

12.1 Pipeline de detecção

O fluxo de detecção foi estruturado para receber frames em tempo real e processá-los de forma sequencial. Cada imagem recebida passa pelo modelo de detecção, que retorna as regiões onde foram identificadas bolas de tênis. A partir dessas regiões, o sistema extrai a posição aproximada de cada bola na imagem e encaminha essa informação para a etapa de conversão geométrica.

De forma resumida, o pipeline atual é composto pelas seguintes etapas:

- recepção do frame enviado pelo módulo de captura;
- aplicação do modelo de detecção de bolas;
- filtragem das detecções com base na confiança do modelo;
- cálculo da posição aproximada de cada bola na imagem;
- conversão das coordenadas em pixels para coordenadas métricas;
- envio das posições detectadas ao módulo responsável pelo estado do sistema.

A Figura 15 representa o fluxo simplificado da detecção das bolas dentro do sistema.

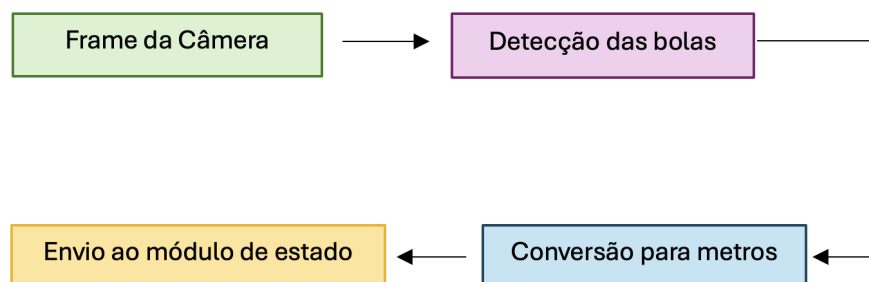


Figura 15: Fluxo simplificado do pipeline de detecção de bolas.

12.2 Modelo de detecção

A detecção das bolas é realizada com base em um modelo YOLO, escolhido por permitir a identificação de objetos diretamente em imagens, com bom equilíbrio entre desempenho e tempo de processamento. Para o desenvolvimento do modelo, foi utilizado o script `VisionTraining.py`, responsável pelo treinamento com o conjunto de dados local.

Esse script é utilizado apenas durante a etapa de desenvolvimento e treinamento, não fazendo parte do pipeline de produção. Durante a operação do sistema, o modelo treinado é carregado pelo módulo de visão computacional, que o aplica aos frames recebidos em tempo de execução.

A utilização de um modelo treinado permite maior robustez em comparação com abordagens baseadas apenas em cor ou forma, especialmente em cenários com variações de iluminação, sombras e objetos visualmente semelhantes às bolas.

12.3 Resultados obtidos

Com a integração do modelo ao pipeline, o sistema passou a detectar as bolas e a disponibilizar as suas posições para o restante sistema.

A Figura 16 apresenta uma visualização de depuração do módulo de visão computacional no ambiente de testes. Além da detecção das bolas de tênis, a imagem mostra também a identificação dos marcadores ArUco associados à frente e à traseira do robô, evidenciando a integração entre a detecção dos objetos de interesse e a estimação visual da orientação da plataforma.

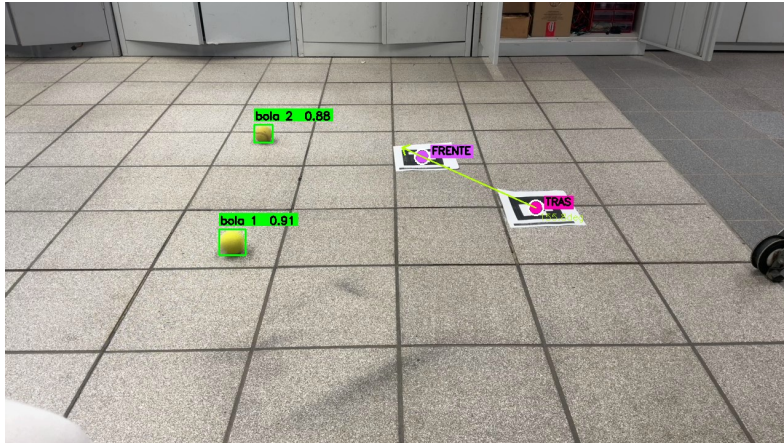


Figura 16: Detecção das bolas e dos marcadores ArUco no ambiente de testes.

A Figura 17 apresenta a imagem resultante da aplicação da homografia, contendo as bolas identificadas pelo modelo.

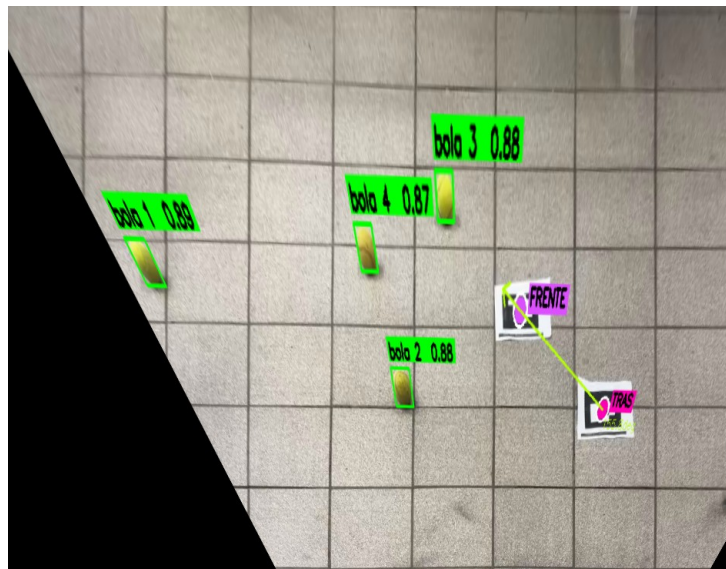


Figura 17: Detecção das bolas no plano retificado após a aplicação da homografia.

Os resultados atuais indicam que o módulo é capaz de identificar bolas em condições controladas e fornecer as suas posições para as etapas seguintes do sistema. Os próximos testes deverão incluir imagens em ambiente real de quadra, permitindo avaliar o desempenho do modelo em condições mais próximas da operação final.

12.4 Integração com o planejamento

Após a detecção e conversão das coordenadas, as posições das bolas são enviadas ao módulo responsável pelo estado do sistema. A partir desse ponto, deixam de ser tratadas apenas como resultados visuais e passam a representar pontos de interesse para a navegação.

O tratamento dessas detecções e a geração da sequência de coleta são descritos com maior detalhe na seção 14.

13 Localização e orientação do robô com ArUco

A localização do robô foi aprofundada nesta etapa através do uso de dois marcadores ArUco. Enquanto a seção 9 apresentou esta solução como um dos avanços gerais da segunda entrega, esta seção descreve a disposição dos marcadores e o método utilizado para estimar a posição e a orientação da plataforma.

Esses referenciais visuais são identificados na imagem capturada pela câmera e utilizados para estimar o estado do robô no plano da quadra. Um dos marcadores é associado à região frontal e o outro à região traseira da plataforma, permitindo formar um eixo de referência para o cálculo da orientação.

13.1 Disposição dos marcadores no robô

Os marcadores ArUco serão fixados na parte superior do robô, de forma visível para a câmera instalada na parede da quadra e orientada com uma inclinação em relação ao plano do chão. Um dos marcadores será alinhado com a parte frontal da plataforma e o outro com a parte traseira. Esta disposição permite formar um eixo de referência longitudinal do robô.

Para garantir maior estabilidade na detecção, está prevista a utilização de uma superfície plana e rígida sobre a estrutura do robô, possivelmente em cartão pluma. Esta superfície servirá como base para fixação dos marcadores, reduzindo deformações e facilitando a sua identificação pela câmera.

A Figura 18 ilustra a disposição prevista dos marcadores ArUco na parte superior do robô.

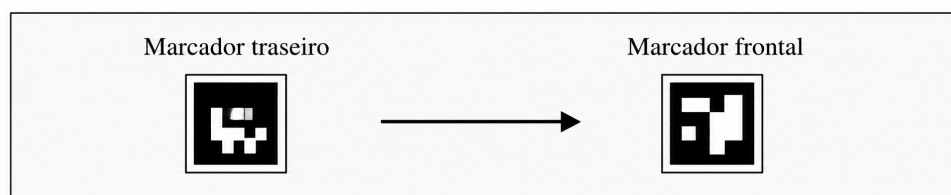


Figura 18: Disposição de exemplo dos marcadores ArUco na parte superior do robô.

13.2 Cálculo da posição e orientação

Após a detecção dos dois marcadores na imagem, as suas coordenadas são convertidas para o referencial métrico da quadra através do módulo de calibração geométrica. Assim, cada marcador passa a ser representado por uma posição (x, y) no plano de operação.

A posição aproximada do robô é calculada a partir do ponto médio entre os marcadores frontal e traseiro. Desta forma, obtém-se uma estimativa do centro da plataforma:

$$x_r = \frac{x_f + x_t}{2} \quad (1)$$

$$y_r = \frac{y_f + y_t}{2} \quad (2)$$

onde (x_r, y_r) representa a posição estimada do robô, (x_f, y_f) representa a posição do marcador frontal e (x_t, y_t) representa a posição do marcador traseiro.

A orientação do robô é obtida a partir do vetor que liga o marcador traseiro ao marcador frontal. O ângulo θ é calculado pela função arco-tangente de duas variáveis:

$$\theta = \arctan 2(y_f - y_t, x_f - x_t) \quad (3)$$

Dessa forma, o sistema passa a conhecer a direção de avanço do robô no mesmo referencial utilizado para representar as bolas e os waypoints de navegação.

Uma visualização experimental da detecção dos marcadores ArUco e da orientação estimada foi apresentada anteriormente na Figura 16, juntamente com os resultados do módulo de visão computacional.

13.3 Integração com o sistema de controle

A informação de posição e orientação do robô é enviada ao módulo de controle, juntamente com o alvo atual definido pelo planejamento. A partir destes dados, o controlador passa a ter as informações necessárias para calcular o erro de posição e orientação em relação ao waypoint desejado.

A utilização de dois marcadores ArUco representa uma solução simples e eficaz para esta fase do projeto, pois permite validar a lógica de localização e controle sem depender inicialmente de sensores adicionais, como encoders ou IMU. Em etapas futuras, estas informações poderão ser combinadas com sensores embarcados para aumentar a robustez da localização durante a operação em quadra.

14 Planejamento de trajetória

Com as posições das bolas e do robô representadas no referencial métrico da quadra, o sistema passa a definir a sequência de pontos que o robô deverá visitar. Esta etapa é realizada pelos módulos responsáveis pela construção do estado do sistema e pelo planejamento da rota de coleta.

Nesta fase, foram consideradas duas estratégias de operação: um modo baseado em faixas e um modo global, no qual a rota é calculada a partir da distribuição das bolas detectadas.

14.1 Construção do estado do sistema

O módulo `GraphProcessor.py` é responsável por receber as detecções provenientes da visão computacional e organizar essas informações em uma representação do estado atual do sistema. Este estado inclui, entre outros dados, a posição estimada do robô, a sua orientação, as bolas detectadas, o alvo atual e a fase de operação.

Para reduzir o impacto de detecções instáveis ou falhas pontuais, o sistema acumula informações ao longo de múltiplos frames antes de gerar uma decisão de planejamento. Dessa forma, evita-se que a rota seja recalculada com base em uma única imagem ruidosa ou em uma detecção momentaneamente incorreta.

Após atingir um nível suficiente de confiança nas detecções, o sistema seleciona a estratégia de operação e disponibiliza o estado atualizado para os módulos seguintes.

14.2 Modos de operação

Foram previstos dois modos de operação para o planejamento da coleta das bolas: o modo por faixas e o modo global.

No modo por faixas, o robô segue regiões pré-definidas da quadra, percorrendo a área de operação de forma estruturada. Esta estratégia pode ser útil em situações em que se pretende uma navegação mais simples e previsível, reduzindo a dependência de um planejamento global completo.

A Figura 19 apresenta um exemplo do modo de planejamento por faixas. Neste modo, a área de operação é dividida em regiões sequenciais, permitindo que o robô percorra a quadra de forma estruturada e previsível.

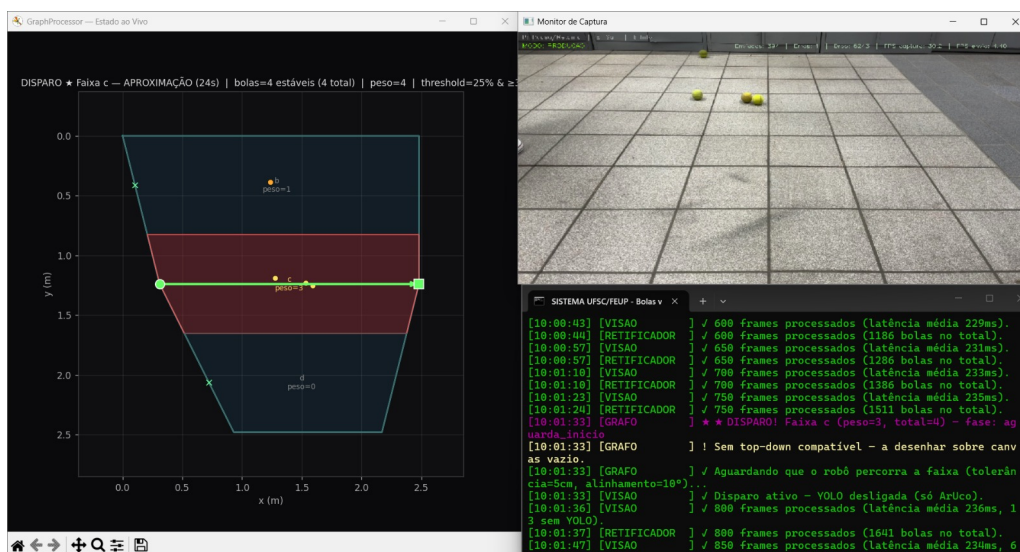


Figura 19: Exemplo de planejamento de coleta por faixas.

No modo global, o sistema utiliza diretamente as posições das bolas detectadas para calcular a sequência de coleta. Neste caso, cada bola é tratada como um waypoint a ser visitado pelo robô. O objetivo é reduzir a distância total percorrida, tornando a coleta mais eficiente.

14.3 Planejamento global da rota

No modo global, o planejamento é realizado pelo módulo `BallCollectionPlanner.py`. Este módulo recebe a lista de bolas detectadas, já representadas em coordenadas métricas, juntamente com a posição atual do robô.

Inicialmente, é gerada uma rota com base no critério do vizinho mais próximo. Nesta abordagem, o próximo ponto escolhido corresponde sempre à bola mais próxima da posição atual do robô ou do último waypoint visitado. Embora este método seja simples e rápido, ele não garante necessariamente a menor rota possível.

Para melhorar a solução inicial, é aplicada uma etapa de otimização do tipo *2-opt*. Esta técnica procura reduzir cruzamentos e encurtar o percurso total através da troca de segmentos da rota. Assim, busca-se obter uma sequência de visita mais eficiente, sem exigir um custo computacional elevado.

A Figura 20 apresenta um exemplo de rota planejada a partir da posição inicial do robô e das bolas detectadas. Neste modo, cada bola é tratada como um waypoint, e a sequência de coleta é definida de forma a reduzir a distância total percorrida.

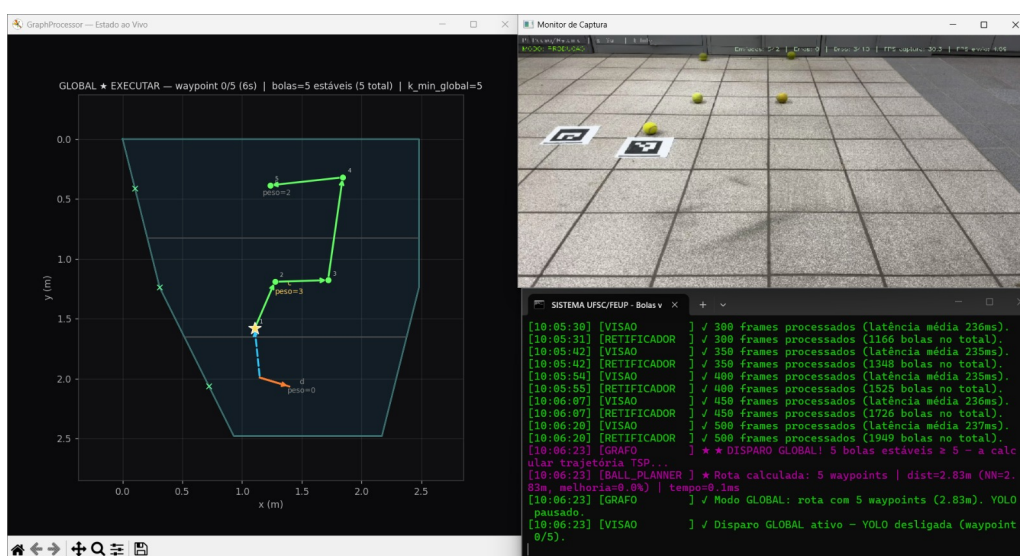


Figura 20: Exemplo de planejamento global da rota de coleta das bolas.

A rota calculada é armazenada internamente pelo planejador, que disponibiliza o waypoint atual para o controlador do robô. À medida que o robô avança e alcança cada ponto da trajetória, o sistema passa para o waypoint seguinte, até que todas as bolas previstas na rota tenham sido visitadas.

Esta abordagem permite que o sistema transforme as detecções visuais em uma sequência de ações de navegação, aproximando a arquitetura implementada do funcionamento esperado para a coleta autônoma das bolas.

15 Controle do robô

Após o planejamento da trajetória, o sistema precisa transformar o waypoint atual em comandos de movimento para o robô. Esta etapa é realizada pelo módulo `RobotController.py`, responsável por

receber o estado atualizado do sistema e calcular as velocidades que devem ser enviadas à plataforma física.

O controle utiliza como entrada a posição e orientação estimadas do robô, obtidas a partir dos marcadores ArUco, e o alvo atual definido pelo módulo de planejamento. A partir destas informações, são calculados o erro de distância até o waypoint e o erro angular entre a direção atual do robô e a direção desejada.

15.1 Entradas do controlador

O módulo `RobotController.py` recebe continuamente as informações disponibilizadas pelo `GraphProcessor.py`. Estas informações descrevem o estado atual do sistema e permitem ao controlador decidir como o robô deve se mover.

As principais entradas utilizadas pelo controlador são:

- posição atual do robô no referencial métrico da quadra;
- orientação atual do robô;
- posição do alvo ou waypoint atual;
- fase de operação do sistema;
- modo operacional selecionado.

Com estes dados, o controlador calcula a direção desejada até o alvo e compara essa direção com a orientação atual do robô. Esta comparação permite determinar se o robô deve primeiro corrigir a sua orientação ou se já pode avançar em direção ao waypoint.

15.2 Estratégia de controle

A estratégia implementada procura evitar que o robô avance quando está demasiado desalinhado em relação ao alvo. Para isso, o sistema utiliza o erro angular como critério principal de decisão.

Quando o erro angular é superior a um limiar definido, o robô prioriza a rotação no próprio eixo, mantendo a velocidade linear nula. Esta lógica permite que a plataforma se alinhe com o waypoint antes de iniciar o deslocamento.

Quando o erro angular se encontra dentro de uma faixa aceitável, o controlador passa a combinar movimento linear e angular. Neste caso, o robô avança em direção ao alvo enquanto realiza pequenas correções de orientação.

De forma simplificada, a lógica utilizada pode ser representada por:

- se o erro angular for elevado, o robô roda no próprio eixo;
- se o erro angular for reduzido, o robô avança em direção ao waypoint;
- durante o avanço, a velocidade angular continua sendo ajustada para corrigir desvios de orientação.

Esta estratégia torna o comportamento do robô mais estável, principalmente durante a aproximação a cada bola ou waypoint da rota planejada.

15.3 Controle angular

O controle da orientação é realizado a partir do erro angular entre a direção atual do robô e a direção desejada. Este erro é utilizado para calcular a velocidade angular ω , responsável pela rotação da plataforma.

Nesta etapa, foi implementado um controle angular baseado em PID. O controlador atua sobre o erro angular, com o objetivo de reduzi-lo ao longo do tempo e alinhar o robô com o alvo definido.

A velocidade linear v é ajustada em função do alinhamento do robô com o waypoint. Quando o erro angular é elevado, a velocidade linear é anulada. Quando o robô está aproximadamente alinhado, a velocidade linear passa a depender da distância até ao alvo e do erro angular, evitando movimentos bruscos em direção a pontos desalinhados.

15.4 Envio de comandos e integração com o protótipo

Após o cálculo das velocidades, o módulo `RobotController.py` prepara os comandos de movimento para envio ao robô físico. A comunicação com a plataforma é realizada através de UDP, tendo como destino o ESP32 responsável pelo acionamento dos motores.

Os comandos enviados incluem, de forma geral, a velocidade linear v e a velocidade angular ω . Estes valores deverão ser interpretados pelo controlador embarcado, que será responsável por convertê-los em sinais adequados para os motores da plataforma.

Durante esta etapa de desenvolvimento, também foi previsto um modo simulado. Neste modo, caso o endereço IP do robô não esteja configurado ou seja inválido, o sistema não envia comandos reais por UDP, mas mantém a lógica de controle em execução. Isto permite testar o comportamento do controlador sem depender da disponibilidade imediata do protótipo físico.

Assim, o módulo de controle estabelece a ligação entre o planejamento de trajetória e a atuação física do robô. Enquanto os módulos anteriores determinam onde estão as bolas e qual deve ser a sequência de coleta, o controlador transforma essa informação em comandos de movimento. A validação completa desta etapa será realizada após a finalização da montagem do protótipo físico e dos testes com o ESP32 e os motores.

A Figura 21 representa o fluxo simplificado do controle implementado.

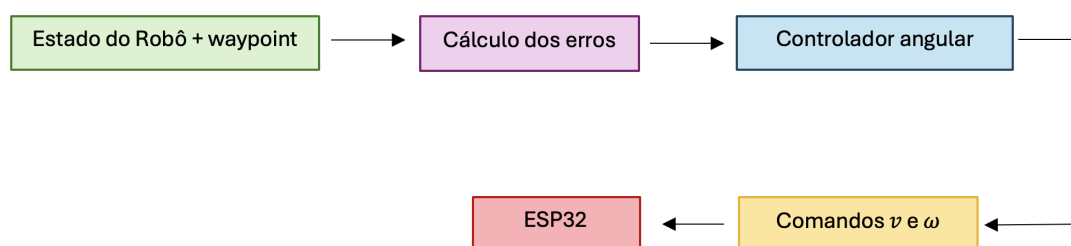


Figura 21: Fluxo simplificado do controle do robô.

16 Configuração e depuração

Além dos módulos principais de visão, planejamento e controle, foram desenvolvidos componentes auxiliares para facilitar a configuração e a depuração do sistema. Estes elementos permitem ajustar parâmetros de operação, acompanhar o comportamento dos módulos e identificar falhas durante os testes.

16.1 Parâmetros configuráveis

O módulo `parametros.py` é responsável por carregar os parâmetros utilizados pelos diferentes componentes do sistema. Estes parâmetros podem ser definidos em um arquivo JSON, permitindo alterar configurações de operação sem modificar diretamente os scripts principais.

Entre os principais parâmetros configuráveis encontram-se:

- endereço IP do robô;
- porta utilizada para comunicação UDP;
- velocidade linear máxima;
- ganhos associados ao controle angular;
- limiar de erro angular utilizado para decidir entre rotação e avanço;
- outros valores relacionados com a operação dos módulos.

No sistema atual, o arquivo de configuração é armazenado no seguinte caminho:

```
resultados/configuracao/parametros.json
```

Caso este arquivo não esteja disponível, o sistema utiliza valores padrão definidos no próprio código. Esta estratégia permite manter a execução funcional mesmo durante fases de teste ou quando a configuração externa ainda não foi criada.

16.2 Ferramentas de suporte ao desenvolvimento

Para acompanhar o funcionamento do sistema durante os testes, foi implementado o módulo `bolas_log.py`, responsável pelo registro centralizado de mensagens. Este módulo permite classificar as mensagens em diferentes níveis, como eventos, avisos, erros, entradas e informações de debug.

Também foi criado o módulo `debug_console.py`, que recebe mensagens de debug enviadas pelos diferentes componentes e as apresenta em um console separado. Esta separação facilita a análise de problemas sem misturar mensagens internas de desenvolvimento com as mensagens principais de operação.

Esses recursos não fazem parte diretamente da lógica de coleta das bolas, mas contribuem para tornar o sistema mais fácil de testar, ajustar e manter durante a integração dos módulos.

17 Execução integrada do sistema

Para facilitar a execução do sistema completo, foi desenvolvido um mecanismo de inicialização responsável por iniciar os principais módulos de software de forma coordenada. Esta abordagem evita que cada processo tenha de ser executado manualmente, reduzindo erros durante os testes e tornando a execução mais simples e reproduzível.

O módulo `MasterControl.py` é responsável por inicializar os subprocessos necessários ao funcionamento do sistema, incluindo os módulos de captura, visão computacional, processamento do estado e controle do robô. Quando o modo de debug está ativo, também é iniciada uma console adicional para visualização das mensagens internas do sistema.

De forma geral, o processo de inicialização envolve os seguintes módulos:

- `imageStreaming.py`, responsável pela captura e envio dos frames;
- `VisionProcessing.py`, responsável pelo processamento de visão computacional;
- `GraphProcessor.py`, responsável pela organização do estado do sistema e planejamento;
- `RobotController.py`, responsável pela geração dos comandos de movimento;
- `debug_console.py`, quando o modo de debug está ativo.

Além disso, foram criados scripts de execução em ambiente Windows. O arquivo `ASTART.bat` inicia o sistema no modo normal de operação, enquanto o arquivo `ASTART_DEBUG.bat` inicia o sistema com suporte adicional à depuração. Antes de uma nova execução, estes scripts encerram processos Python anteriores que possam ter permanecido ativos, evitando conflitos entre execuções sucessivas, especialmente nas portas de comunicação utilizadas pelos módulos.

Com esta organização, a execução integrada do sistema torna-se mais controlada, permitindo repetir testes com maior facilidade e preparar a integração progressiva entre captura, visão, planejamento, controle e robô físico.

18 Montagem do protótipo físico

Paralelamente ao desenvolvimento dos módulos de software, iniciou-se a montagem do protótipo físico do robô coletor. Esta etapa tem como objetivo preparar a plataforma que será utilizada para validar, em ambiente real, os módulos de localização, planejamento, controle e recolha das bolas.

Nesta fase, a montagem ainda se encontra em andamento. Os principais componentes necessários à construção da plataforma já foram reunidos, e a estrutura do robô está sendo preparada para receber os elementos mecânicos, eletrônicos e visuais necessários à operação do sistema.

18.1 Estado atual da montagem

O protótipo físico será composto por uma plataforma móvel, sistema de acionamento dos motores, controlador embarcado e estrutura de suporte para os marcadores ArUco. O ESP32 será utilizado para receber os comandos enviados pelo módulo de controle e converter esses comandos em sinais adequados para os motores.

Atualmente, está sendo realizada a ligação das pontes H ao ESP32 e ao sistema de alimentação do robô. Nesta fase de testes, embora a alimentação final esteja prevista para ser feita por bateria, está sendo utilizada uma fonte de tensão externa para validar as conexões elétricas e o acionamento dos motores de forma mais controlada.

A Figura 22 apresenta o estado atual da montagem do protótipo físico.



Figura 22: Estrutura física do protótipo móvel utilizado para a integração do sistema.

A Figura 23 apresenta a montagem preliminar dos componentes eletrônicos utilizados no controle do robô. Nesta etapa, os elementos ainda se encontram organizados em bancada, permitindo testar individualmente as ligações entre o ESP32, os drivers dos motores e os restantes componentes antes da fixação definitiva na estrutura do protótipo.

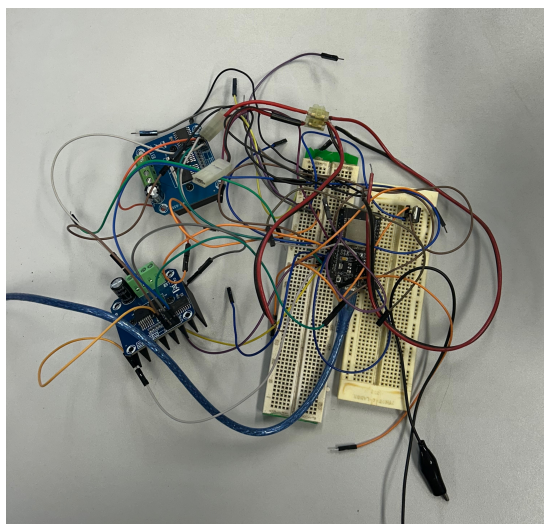


Figura 23: Montagem preliminar dos componentes eletrônicos, ainda em fase de organização e integração.

18.2 Integração prevista com o software

A integração entre o software desenvolvido e o protótipo físico será realizada através da comunicação entre o módulo `RobotController.py` e o ESP32. O controlador em software calcula as velocidades linear e angular desejadas, enquanto o ESP32 deverá interpretar esses comandos e atuar sobre os motores da plataforma.

Após a conclusão da montagem, serão realizados testes progressivos de integração. Inicialmente, pretende-se validar a comunicação UDP entre o computador e o ESP32. Em seguida, serão testados os comandos básicos de movimento, a resposta dos motores, a detecção dos marcadores ArUco sobre o robô e a capacidade do sistema de deslocar a plataforma em direção a waypoints definidos.

Esta etapa será fundamental para verificar se o comportamento previsto em software corresponde ao movimento real da plataforma. A partir desses testes, poderão ser ajustados parâmetros de controle, limites de velocidade e estratégias de navegação, preparando o sistema para a validação em ambiente de quadra.

19 Testes realizados e validação parcial

Ao longo da segunda etapa, foram realizados testes parciais dos principais módulos implementados. Como o protótipo físico ainda se encontra em montagem, a validação completa do sistema em ambiente real ainda não foi concluída. No entanto, foi possível verificar o funcionamento individual e integrado de vários componentes de software.

A Tabela 3 resume os testes realizados até o momento e o estado atual de cada módulo.

Módulo	Teste realizado	Estado atual
Calibração geométrica	Seleção de quatro pontos de referência, cálculo da homografia e conversão de coordenadas de pixels para metros.	Implementado e integrado ao pipeline de visão.
Detecção de bolas	Aplicação do modelo YOLO aos frames capturados e identificação das bolas presentes na imagem.	Funcional em condições controladas, com necessidade de validação adicional em quadra real.
Localização do robô	Detecção de dois marcadores ArUco e cálculo da posição e orientação do robô.	Implementado em software, aguardando validação sobre o protótipo físico.
Planejamento de trajetória	Geração de rota global a partir das posições das bolas detectadas, utilizando vizinho mais próximo e otimização <i>2-opt</i> .	Implementado e preparado para integração com o controle.
Controle do robô	Cálculo de comandos de velocidade linear e angular a partir do waypoint atual e da orientação do robô.	Implementado em modo de software, aguardando testes completos com o ESP32 e motores.
Comunicação entre módulos	Troca de mensagens entre captura, visão, processamento do estado e controle.	Implementada com portas locais e mecanismo de <i>handshake</i> entre captura e visão.
Protótipo físico	Montagem da plataforma, preparação dos componentes e definição da posição dos marcadores ArUco.	Em andamento.

Tabela 3: Resumo dos testes realizados e estado atual de validação dos módulos.

Os testes realizados indicam que os principais módulos de software já se encontram implementados e preparados para a integração. A principal etapa pendente é a validação experimental com o robô físico. Esta validação permitirá avaliar a resposta dos motores, a comunicação com o ESP32, a estabilidade da localização por ArUco, a capacidade do sistema de seguir waypoints em ambiente real e a cadência de transmissão de informação até o robô.

Assim, a validação parcial indica o avanço do projeto em relação à primeira entrega, mas também evidencia que os próximos esforços devem concentrar-se na integração física e nos testes em quadra.

20 Cronogramas da segunda entrega

Nesta seção são apresentados os cronogramas atualizados referentes à segunda etapa do projeto. O primeiro cronograma resume as atividades realizadas entre a primeira e a segunda entrega, enquanto o segundo apresenta o planejamento das próximas atividades até ao final do semestre.

20.1 Atividades realizadas entre a primeira e a segunda entrega

A Figura 24 apresenta o conjunto de atividades desenvolvidas desde a entrega da primeira versão do relatório até a segunda entrega. Neste período, o trabalho concentrou-se principalmente na implementação dos módulos de software, na integração do pipeline de visão computacional, na calibração geométrica, na localização do robô por ArUco, no planejamento de trajetória e no desenvolvimento inicial do controlador.

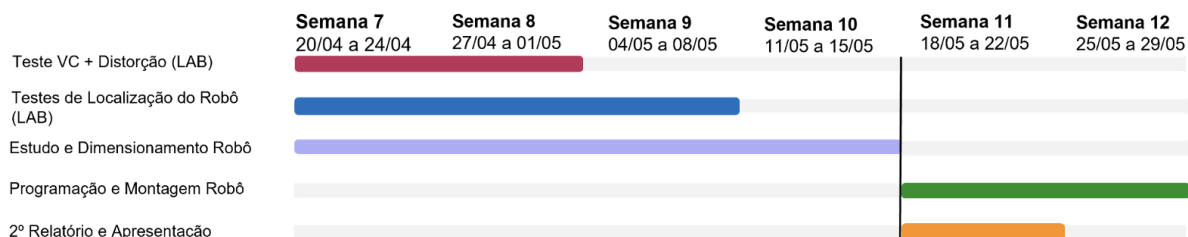


Figura 24: Cronograma das atividades realizadas entre a primeira e a segunda entrega.

20.2 Atividades previstas até a entrega final

A Figura 25 apresenta o planejamento das atividades previstas para as semanas restantes do semestre. As próximas etapas concentram-se na finalização da integração com o protótipo físico, validação da comunicação com o ESP32, testes dos motores, ajuste dos parâmetros de controle, realização de testes em quadra e preparação da demonstração final.

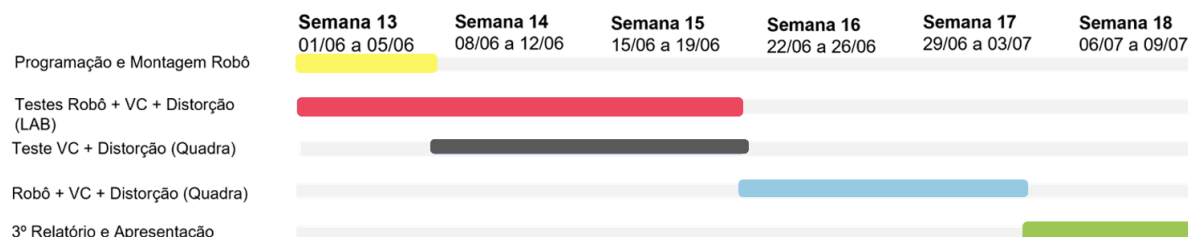


Figura 25: Cronograma das atividades previstas até à entrega final do projeto.

De forma geral, os cronogramas evidenciam a evolução do projeto desde a fase de validação preliminar até à implementação integrada dos principais módulos. O planejamento futuro poderá ser ajustado conforme os resultados obtidos durante os testes com o protótipo físico e em ambiente real de quadra.

21 Dificuldades encontradas

Durante esta segunda etapa, foram identificadas algumas dificuldades técnicas associadas à implementação e integração dos módulos do sistema.

Uma das principais dificuldades foi garantir que as coordenadas obtidas na imagem fossem convertidas de forma coerente para o referencial métrico da quadra. Como a câmera observa a área de operação a partir de uma posição superior, pequenas alterações na posição da câmera ou na escolha dos pontos de referência podem afetar a homografia e, consequentemente, a precisão das coordenadas calculadas.

A localização do robô por marcadores ArUco também exigiu atenção especial. Para que a orientação seja calculada corretamente, os dois marcadores precisam permanecer visíveis para a câmera e estar fixados de forma estável na parte superior da plataforma. A montagem física do robô deverá garantir que estes marcadores não sofram deslocamentos ou inclinações significativas durante o movimento.

Por fim, a integração com o protótipo físico ainda representa uma etapa crítica. Embora a lógica de controle e o envio de comandos ao ESP32 já estejam previstos em software, será necessário validar a resposta real dos motores, ajustar parâmetros de velocidade e verificar se o comportamento físico da plataforma corresponde ao esperado pelo sistema de controle.

De forma geral, as dificuldades encontradas contribuíram para orientar as próximas etapas do projeto, indicando os pontos que exigem maior atenção durante a integração e validação em ambiente real.

22 Próximas etapas

Após os avanços obtidos nesta segunda entrega, as próximas etapas estarão focadas principalmente na integração do software com o protótipo físico e na validação do sistema em ambiente real.

As principais atividades previstas são:

- finalizar a montagem mecânica e eletrônica do protótipo;
- fixar os marcadores ArUco na parte superior do robô;
- validar a comunicação UDP entre o computador e o ESP32;
- testar os comandos básicos de movimento;
- ajustar os parâmetros de controle, como velocidade máxima, ganho angular e limiar de erro angular;
- validar a localização do robô por ArUco com a plataforma em movimento;
- integrar o planejamento de trajetória com o deslocamento real do robô;
- realizar testes de seguimento de waypoints;
- testar a detecção das bolas em ambiente real de quadra;
- avaliar o desempenho do sistema completo durante a coleta das bolas;
- melhorar o retificador para a correção de erros.

Inicialmente, os testes serão realizados com trajetórias simples e poucos waypoints. Em seguida, pretende-se aumentar gradualmente a complexidade, incluindo mais bolas e avaliando a estabilidade da localização, do planejamento e do controle.

Por fim, os resultados obtidos deverão ser comparados com os requisitos definidos anteriormente, permitindo identificar limitações do sistema e propor melhorias para a versão final do protótipo.

23 Conclusão da segunda entrega

A segunda entrega representou um avanço importante no desenvolvimento do robô coletor de bolas. Nesta etapa, o projeto passou de uma fase principalmente conceitual para uma fase de implementação e integração dos principais módulos de software.

Foram desenvolvidos e integrados módulos de visão computacional, calibração geométrica, localização do robô com ArUco, planejamento de trajetória e controle. Com isso, o sistema já consegue processar imagens, identificar bolas e robô, converter posições para coordenadas métricas e gerar informações úteis para a navegação.

Também foi iniciada a montagem do protótipo físico, que será essencial para validar o comportamento do sistema em ambiente real. A comunicação com o ESP32 e a lógica de controle já se encontram preparadas para essa próxima etapa.

Apesar de a validação completa ainda depender dos testes com o robô físico, os resultados obtidos até o momento demonstram que o projeto evoluiu de forma consistente e que a base técnica para a continuação do desenvolvimento já está estabelecida.

Dessa forma, a segunda entrega consolida a base técnica necessária para a etapa final do projeto, na qual a prioridade será validar experimentalmente o comportamento do sistema integrado com o protótipo físico em ambiente real de operação.

Referências

- [1] Drillbot, "Drillbot – Soluções para treino de tênis", Disponível em: <http://drillbot.net>.
- [2] BOCHKOVSKIY, Alexey; WANG, Chien-Yao; LIAO, Hong-Yuan Mark. *YOLOv4: Optimal Speed and Accuracy of Object Detection*. arXiv preprint arXiv:2004.10934, 2020.
- [3] HARTLEY, Richard; ZISSERMAN, Andrew. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2004.
- [4] SANYAL, Ayush; GHOSH, Sourav; MISHRA, Akash; GHOSH, Priya. Design and Development of a Tennis Ball Collecting Robot. *International Journal of Engineering Research and Technology*, v. 6, n. 05, 2017.
- [5] OPENCV. Camera Calibration and 3D Reconstruction. Disponível em: <https://docs.opencv.org/>.
- [6] OPENCV. ArUco marker detection. Disponível em: <https://docs.opencv.org/>.
- [7] ULTRALYTICS. YOLOv8 Documentation. Disponível em: <https://docs.ultralytics.com/models/yolov8/>.